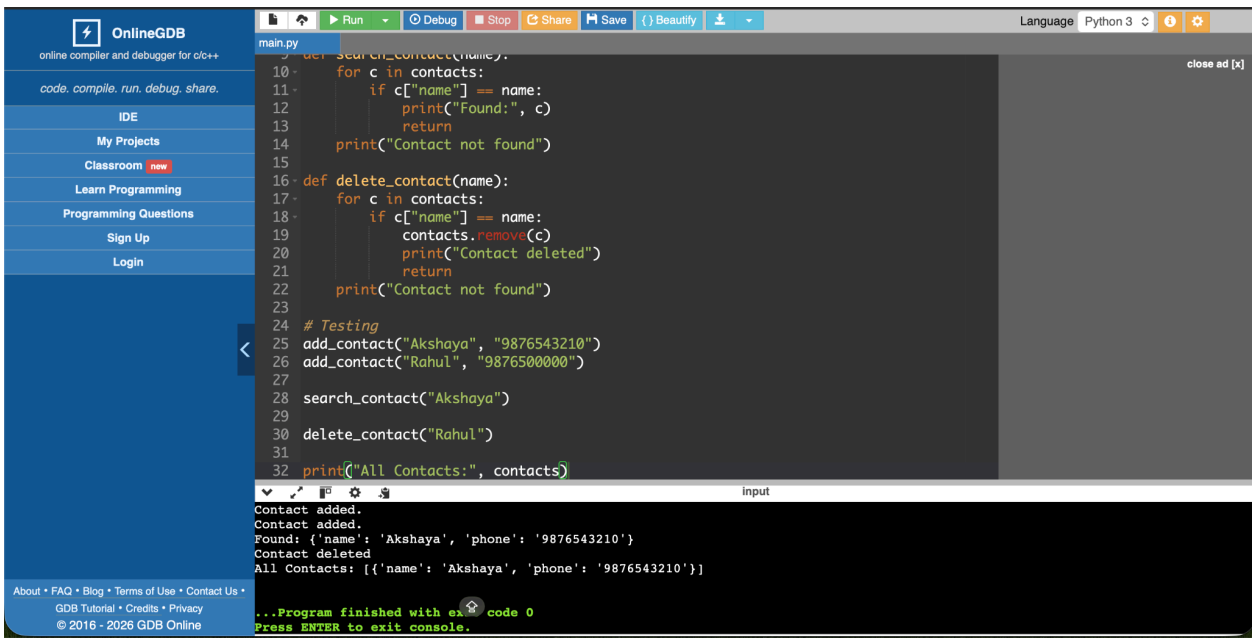
2303A51563
BATCH=10
Task 1: Smart Contact Manager
CODE

```
# Contact Manager using Array (List)

contacts = []

def add_contact(name, phone):
    contacts.append({"name": name, "phone": phone})
    print("Contact added.")

def search_contact(name):
    for c in contacts:
        if c["name"] == name:
            print("Found:", c)
            return
    print("Contact not found")

def delete_contact(name):
    for c in contacts:
        if c["name"] == name:
            contacts.remove(c)
            print("Contact deleted")
            return
    print("Contact not found")

# Testing
add_contact("Akshaya", "9876543210")
add_contact("Rahul", "9876500000")

search_contact("Akshaya")

delete_contact("Rahul")

print("All Contacts:", contacts)
```



2. Linked List Implementation

CODE

```
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

class ContactList:
    def __init__(self):
        self.head = None
```

```
    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node
        print("Contact added")

    def search_contact(self, name):
        temp = self.head
        while temp:
            if temp.name == name:
                print("Found:", temp.name, temp.phone)
                return
            temp = temp.next
        print("Contact not found")

    def delete_contact(self, name):
        temp = self.head
        prev = None

        while temp:
            if temp.name == name:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                print("Deleted")
                return
            prev = temp
            temp = temp.next
        print("Contact not found")


# Testing
cl = ContactList()
cl.add_contact("Akshaya", "9876543210")
cl.add_contact("Rahul", "9876500000")

cl.search_contact("Rahul")

cl.delete_contact("Akshaya")
```

OnlineGDB
online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login

Run | Debug | Stop | Share | Save | { } Beautify | Language Python 3

main.py

close ad [x]

```python
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

class ContactList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node
        print("Contact added")

    def search_contact(self, name):
        temp = self.head
        while temp:
            if temp.name == name:
                print("Found:", temp.name, temp.phone)
                return
            temp = temp.next
```

input

```
Contact added
Contact added
Found: Rahul 9876500000
Deleted


...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

OnlineGDB
online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login

Run | Debug | Stop | Share | Save | { } Beautify | Language Python 3

main.py

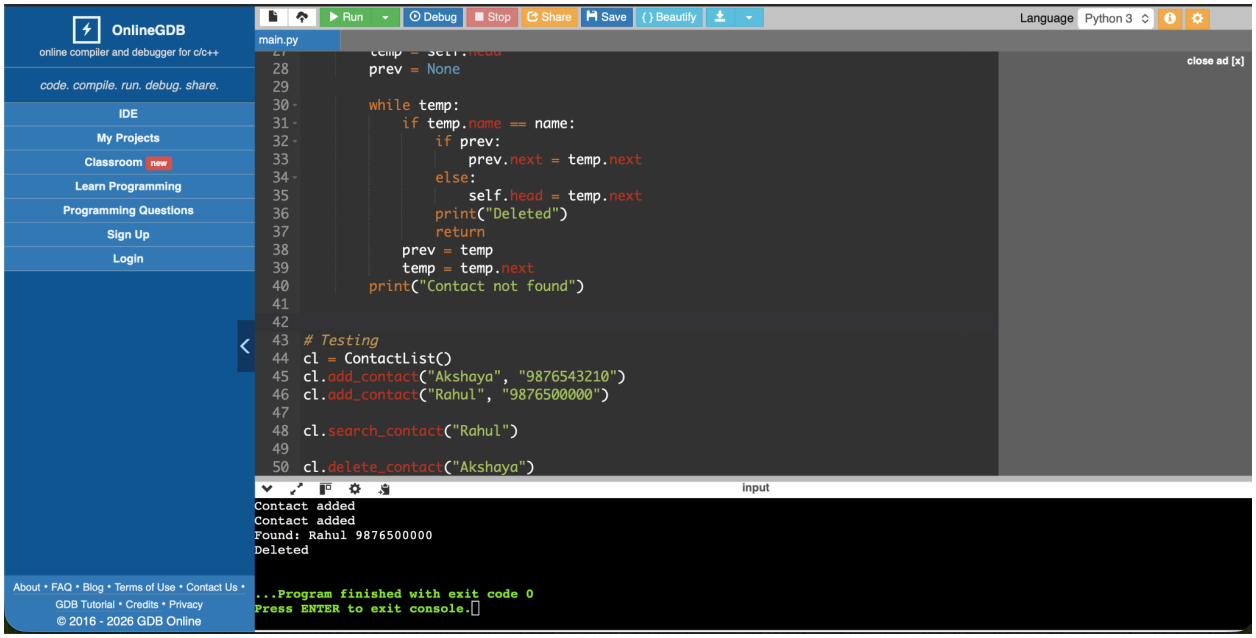
close ad [x]

```python
        print("Contact not found")

    def delete_contact(self, name):
        temp = self.head
        prev = None

        while temp:
            if temp.name == name:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                print("Deleted")
                return
            prev = temp
            temp = temp.next
        print("Contact not found")


# Testing
cl = ContactList()
cl.add_contact("Akshaya", "9876543210")
cl.add_contact("Rahul", "9876500000")
```

input

```
Contact added
Contact added
Found: Rahul 9876500000
Deleted


...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online



Task 3: Emergency Help Desk (Stack)

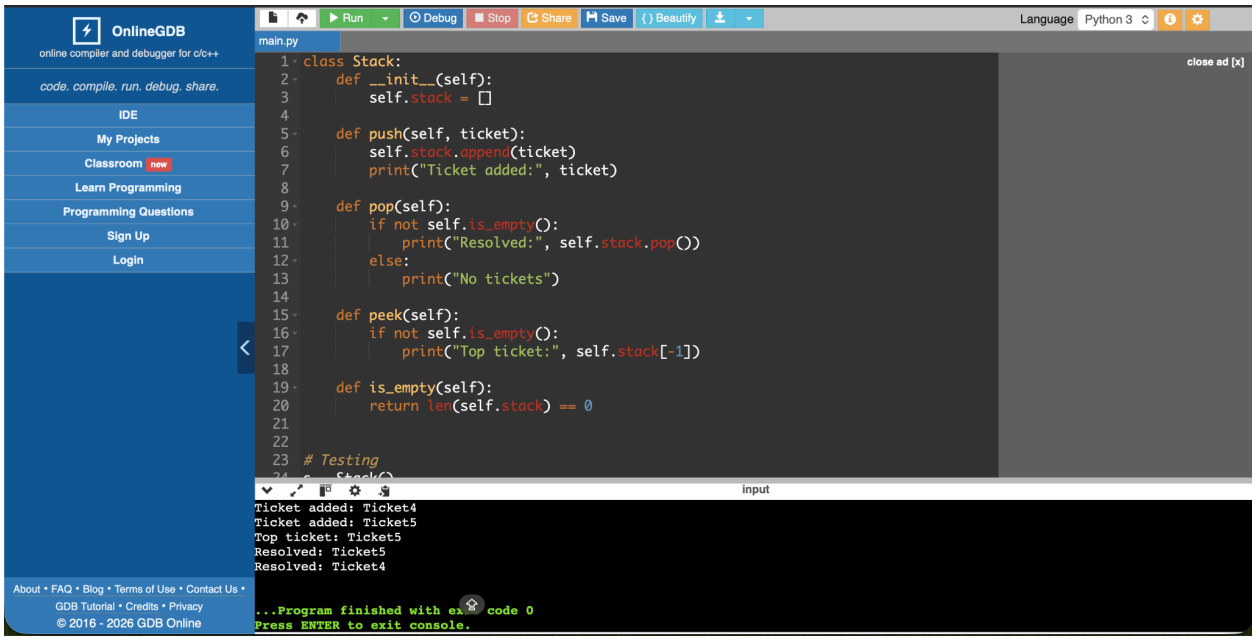
CODE

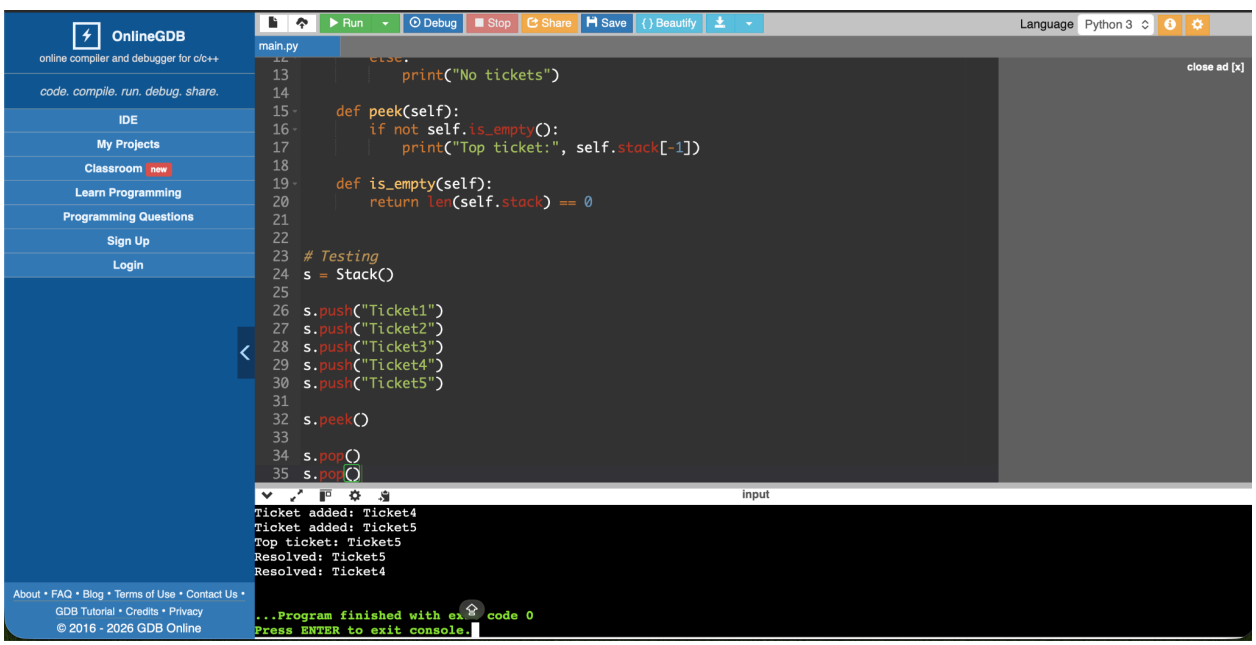
```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, ticket):
        self.stack.append(ticket)
        print("Ticket added:", ticket)

    def pop(self):
        if not self.is_empty():
            print("Resolved:", self.stack.pop())
        else:
            print("No tickets")

    def peek(self):
        if not self.is_empty():
            print("Top ticket:", self.stack[-1])

    def is_empty(self):
        return len(self.stack) == 0


# Testing
s = Stack()

s.push("Ticket1")
```

s.push("Ticket2")
s.push("Ticket3")
s.push("Ticket4")
s.push("Ticket5")

s.peek()

s.pop()
s.pop()





Task 4: Hash Table with Chaining
CODE

```python
class HashTable:

    def __init__(self):
        self.size = 10
        self.table = [[] for _ in range(self.size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        self.table[index].append((key, value))

    def search(self, key):
        index = self.hash_function(key)

        for k, v in self.table[index]:
            if k == key:
                return v
        return "Not found"

    def delete(self, key):
        index = self.hash_function(key)

        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index].pop(i)
                print("Deleted")
                return


# Testing
h = HashTable()

h.insert("Akshaya", "9876543210")
h.insert("Rahul", "9999999999")

print(h.search("Akshaya"))

h.delete("Rahul")
```

OnlineGDB
online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login

Run | Debug | Stop | Share | Save | { } Beautify | Language Python 3

main.py

close ad [x]

```python
class HashTable:

    def __init__(self):
        self.size = 10
        self.table = [[] for _ in range(self.size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        self.table[index].append((key, value))

    def search(self, key):
        index = self.hash_function(key)

        for k, v in self.table[index]:
            if k == key:
                return v
        return "Not found"

    def delete(self, key):
        index = self.hash_function(key)
```

input

```
9876543210
Deleted


...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

OnlineGDB
online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login

Run | Debug | Stop | Share | Save | { } Beautify | Language Python 3

main.py

close ad [x]

```python
            if k == key:
                return v
        return "Not found"

    def delete(self, key):
        index = self.hash_function(key)

        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index].pop(i)
                print("Deleted")
                return


# Testing
h = HashTable()

h.insert("Akshaya", "9876543210")
h.insert("Rahul", "9999999999")

print(h.search("Akshaya"))

h.delete("Rahul")
```

input

```
9876543210
Deleted


...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online